

Q.1 Demonstration of Grid Search and Random Search Hyperparameter Tuning

1. Dataset Selection and Model Choice

Dataset: [FIFA World Cup 2022 Tweets dataset](https://www.kaggle.com/datasets/tirendazacademy/fifa-world-cup-2022-tweets)

(<https://www.kaggle.com/datasets/tirendazacademy/fifa-world-cup-2022-tweets>)

Task: Sentiment Classification:- Train a logistic regression model to classify tweets into one of these sentiment categories (multiclass text classification) .

Why Logistic Regression?

- Widely used for text classification tasks.
- Works well with TF-IDF features extracted from text.
- Provides interpretable results and is computationally efficient compared to more complex models.

Before training, I applied text cleaning steps such as converting to lowercase, removing URLs, mentions, hashtags, punctuation, and numbers. This ensured that the input text was consistent and free from noise.

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import re
import string

from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from scipy.stats import uniform

# 2. Load Dataset
df = pd.read_csv("/content/fifa_world_cup_2022_tweets.csv")
print(df.head())
print(df.info())
```

Unnamed: 0	Date Created	Number of Likes
0	2022-11-20 23:59:21+00:00	4
1	2022-11-20 23:59:01+00:00	3
2	2022-11-20 23:58:41+00:00	1
3	2022-11-20 23:58:33+00:00	1
4	2022-11-20 23:58:28+00:00	0

Source of Tweet	Tweet
0 Twitter Web App	What are we drinking today @TucanTribe \n@MadB...
1 Twitter for iPhone	Amazing @CanadaSoccerEN #WorldCup2022 launch ...
2 Twitter for iPhone	Worth reading while watching #WorldCup2022 htt...
3 Twitter Web App	Golden Maknae shining bright\n\nhttps://t.co/...
4 Twitter for Android	If the BBC cares so much about human rights, h...

Sentiment

0	neutral
1	positive
2	positive
3	positive
4	negative

What can I help you build?

Dataset Loading

2. Features, Target, and Pipeline

- Feature (X): The cleaned tweet text.
- Target (y): The sentiment label (positive, neutral, negative).

To process text and train the model efficiently, I defined a Pipeline consisting of two steps:

- TF-IDF Vectorization – converts text into numerical features while considering word importance across documents.
- Logistic Regression Classifier – predicts the sentiment using the TF-IDF features.

The pipeline allows hyperparameter tuning to be applied seamlessly on both preprocessing (TF-IDF) and the model (Logistic Regression).

```
# 3. Basic Cleaning
def clean_text(text):
    # Lowercase
    text = text.lower()
    # Remove URLs
    text = re.sub(r"http\S+|www\S+|https\S+", '', text, flags=re.MULTILINE)
    # Remove mentions and hashtags
    text = re.sub(r'@\w+|#', '', text)
    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))
    # Remove numbers
    text = re.sub(r'\d+', '', text)
    return text

df['clean_tweet'] = df['Tweet'].astype(str).apply(clean_text)

# 4. Define Features & Target
X = df['clean_tweet']
y = df['Sentiment'] # (Positive / Neutral / Negative)

# Train-test split (Stratified to keep sentiment distribution)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

print("Train size:", X_train.shape[0], " Test size:", X_test.shape[0])

# 5. Build Pipeline
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=10000, stop_words='english')),
    ('clf', LogisticRegression(max_iter=1000, random_state=42))
])
```

Train size: 18019 Test size: 4505

Cleaning dataset and extraction of features into vectors using TF-IDF vectorization

3. Grid Search Hyperparameter Tuning

GridSearchCV performs an exhaustive search over all possible hyperparameter combinations provided in a grid. **For this task, the following hyperparameters were tuned:**

- `tfidf__ngram_range`: whether to consider single words or word pairs.
- `tfidf__max_df`: maximum document frequency threshold to filter common words.
- `clf__C`: regularization strength for Logistic Regression.
- `clf__penalty`: type of penalty (here, L2).
- `clf__class_weight`: to handle class imbalance.

GridSearchCV trained the model on all combinations using 5-fold cross validation. It selected the parameters that gave the best average macro F1 score.

RESULTS:

```
# 6. Hyperparameter Tuning - GridSearchCV
param_grid = {
    'tfidf__ngram_range': [(1,1), (1,2)],
    'tfidf__max_df': [0.75, 1.0],
    'clf__C': [0.01, 0.1, 1, 10],
    'clf__penalty': ['l2'],
    'clf__class_weight': [None, 'balanced']
}

grid_search = GridSearchCV(
    pipeline, param_grid,
    cv=5, scoring='f1_macro',
    n_jobs=-1, verbose=2
)

grid_search.fit(X_train, y_train)

print("\nBest Parameters (GridSearch):", grid_search.best_params_)
print("Best CV Score (GridSearch):", grid_search.best_score_)

Fitting 5 folds for each of 32 candidates, totalling 160 fits

Best Parameters (GridSearch): {'clf__C': 1, 'clf__class_weight': 'balanced', 'clf__penalty': 'l2', 'tfidf__max_df': 0.75, 'tfidf__ngram_range': (1, 1)}
Best CV Score (GridSearch): 0.7073142887141951
```

Best Hyperparameter obtained from Grid Search Hyperparameter fine tuning

Best Parameters (Grid Search):

```
{
'clf__C': 1, 'clf__class_weight': 'balanced',
'clf__penalty': 'l2',
'tfidf__max_df': 0.75,
'tfidf__ngram_range': (1, 1)
}
```

Best CV Score: 0.7073 [average cross validation score]

When evaluated on the test set, GridSearch achieved:

Accuracy: 71%

Macro F1 Score: 0.72

```
# 7. Evaluation on Test Set for GridSearch best model hyperparameters
y_pred_grid = grid_search.predict(X_test)
print("\nClassification Report (GridSearch):\n", classification_report(y_test, y_pred_grid))
print("\nConfusion Matrix (GridSearch):\n", confusion_matrix(y_test, y_pred_grid))
```

```
Classification Report (GridSearch):
              precision    recall  f1-score   support

   negative      0.69      0.75      0.72      1157
    neutral      0.68      0.68      0.68      1650
    positive      0.78      0.73      0.75      1698

 accuracy              0.71      4505
 macro avg      0.71      0.72      0.72      4505
weighted avg      0.72      0.71      0.72      4505
```

```
Confusion Matrix (GridSearch):
[[ 873  190   94]
 [ 273 1115  262]
 [ 121  345 1232]]
```

Evaluation on test set after Grid Search Hyperparameter finetuning

Stronger performance on positive and negative tweets compared to neutral tweets.

4. Randomized Search Hyperparameter Tuning

RandomizedSearchCV performs a random sampling of hyperparameter combinations from specified distributions, instead of exhaustively checking all options. It is more efficient for larger search spaces.

For this task, **RandomizedSearchCV** tuned the same kinds of parameters but drew values randomly for:

- `tfidf__max_df` (sampled from a uniform range 0.5–1.0).
- `clf__C` (sampled from a uniform range 0.01–10).
- Other options such as `n-gram` range and class weights.

It ran for 20 random combinations with 5-fold cross validation, again optimizing the macro F1 score.

RESULTS

Best Parameters (Randomized Search):

`{'clf__C': 1.84, 'clf__class_weight': 'balanced', 'tfidf__max_df': 0.8058, 'tfidf__ngram_range': (1, 1)}`

Best CV Score: 0.7072

```
# 8. Hyperparameter Tuning - RandomizedSearchCV
param_dist = {
    'tfidf__ngram_range': [(1,1), (1,2)],
    'tfidf__max_df': uniform(0.5, 0.5), # values between 0.5-1.0
    'clf__C': uniform(0.01, 10),
    'clf__class_weight': [None, 'balanced']
}

random_search = RandomizedSearchCV(
    pipeline, param_distributions=param_dist,
    n_iter=20, cv=5, scoring='f1_macro',
    random_state=42, n_jobs=-1, verbose=2
)

random_search.fit(X_train, y_train)

print("\nBest Parameters (RandomizedSearch):", random_search.best_params_)
print("Best CV Score (RandomizedSearch):", random_search.best_score_)

Fitting 5 folds for each of 20 candidates, totalling 100 fits

Best Parameters (RandomizedSearch): {'clf__C': np.float64(1.8440450985343382), 'clf__class_weight': 'balanced', 'tfidf__max_df': np.float64(0.8058265802441404),
Best CV Score (RandomizedSearch): 0.7071941959754724
```

Best Hyperparameter obtained from Randomized Search Hyperparameter fine tuning

On the test set, Randomized Search achieved:

Accuracy: 72%

Macro F1 Score: 0.72

Slight improvement compared to Grid Search, particularly for the positive class.

```
# 9. Evaluation on Test Set (Randomized best model)
y_pred_rand = random_search.predict(X_test)
print("\nClassification Report (RandomizedSearch):\n", classification_report(y_test, y_pred_rand))
print("\nConfusion Matrix (RandomizedSearch):\n", confusion_matrix(y_test, y_pred_rand))
```

```
Classification Report (RandomizedSearch):
              precision    recall  f1-score   support

negative      0.70      0.74      0.72      1157
neutral       0.68      0.68      0.68      1650
positive      0.78      0.74      0.76      1698

accuracy              0.72      4505
macro avg          0.72      0.72      0.72      4505
weighted avg       0.72      0.72      0.72      4505
```

```
Confusion Matrix (RandomizedSearch):
[[ 861  204   92]
 [ 261 1120  269]
 [ 114  332 1252]]
```

Evaluation on test set after RandomizedSearch Hyperparameter finetuning